

Jules, create repository abisheakp197/nti-badge. This is the NTI-1 Verified Badge — a federated, zero-lock-in compliance certificate system for AI agent projects.

DIFFERENTIATORS FROM ALL EXISTING BADGE SYSTEMS:

1. Certificates are Dilithium5 post-quantum signed (no giant uses PQC for badges)
2. Multi-OIDC provider support (GitHub, GitLab, Google, Bitbucket, custom)
3. Portable NTI-Cert v1 JSON format — user owns it forever, verifiable offline
4. Federated protocol — any org can run a mirror node (no central authority)
5. Zero data lock-in — registry indexes cert hashes only, never full data
6. Works across ecosystems — GitHub, GitLab, Bitbucket, Gitea, self-hosted
7. Git-linked audit chain — every cert bound to a commit SHA
8. Enterprise SSO + private federation
9. Compliance tiers (Bronze/Silver/Gold/Platinum)
10. Optional zero-knowledge score proofs for confidential repos

REPO STRUCTURE:

```
nti-badge/
├── action.yml
├── entrypoint.sh
├── api/
│   ├── badge.py
│   ├── verify.py
│   ├── registry.py
│   ├── leaderboard.py
│   ├── certificate.py
│   ├── transparency.py
│   ├── org.py
│   └── federation.py
├── lib/
│   ├── __init__.py
│   ├── oidc.py
│   ├── store.py
│   ├── crypto.py
│   ├── certificate.py
│   ├── drift.py
│   ├── tiers.py
│   └── spec.py
├── public/
│   ├── index.html
│   └── repo.html
├── docs/
│   └── NTI-CERT-SPEC.md
├── vercel.json
└── requirements.txt
```

- |— README.md
- |— LICENSE
- |— .gitignore

=== FILE 1: action.yml ===

name: 'NTI Verify'

description: 'Generate a Dilithium5-signed NTI-1 compliance certificate for your AI agent repo.
Portable, offline-verifiable, no lock-in.'

author: 'Abisheak'

branding:

icon: 'shield'

color: 'green'

inputs:

fail-below:

description: 'Fail if score is below this value (0-100)'

required: false

default: '0'

submit:

description: 'Index cert hash + score in the public registry (optional)'

required: false

default: 'true'

drift-fail:

description: 'Fail if score dropped since last verified cert'

required: false

default: 'false'

registry-url:

description: 'Registry endpoint. Default is public NTI registry. Change for federated deployment.'

required: false

default: 'https://nti-badge.vercel.app'

zk-threshold:

description: 'If set, publish a zero-knowledge proof that score $\geq$ threshold without revealing exact score'

required: false

default: '0'

runs:

using: 'composite'

steps:

- name: Set up Python

uses: actions/setup-python@v5

with:

python-version: '3.11'

- name: Install tools
shell: bash
run: pip install nti-scanner ube-foundation

- name: Run NTI scan
id: scan
shell: bash
run: |
set +e
nti-scanner scan . --format json --output nti-report.json
if [! -f nti-report.json]; then
echo "score=0" >> \$GITHUB_OUTPUT
exit 0
fi
SCORE=\$(python -c "import json; print(json.load(open('nti-report.json'))['nti1_score'])")
FINDINGS=\$(python -c "import json; print(len(json.load(open('nti-report.json'))['findings']))")
echo "score=\$SCORE" >> \$GITHUB_OUTPUT
echo "findings=\$FINDINGS" >> \$GITHUB_OUTPUT
echo "NTI-1 Score: \$SCORE/100 (\$FINDINGS findings)"
continue-on-error: true

- name: Determine compliance tier
id: tier
shell: bash
run: |
SCORE=\${{ steps.scan.outputs.score }}
if ["\$SCORE" -ge 95]; then TIER="Platinum"
elif ["\$SCORE" -ge 85]; then TIER="Gold"
elif ["\$SCORE" -ge 70]; then TIER="Silver"
elif ["\$SCORE" -ge 50]; then TIER="Bronze"
else TIER="Unranked"; fi
echo "tier=\$TIER" >> \$GITHUB_OUTPUT
echo "Compliance tier: \$TIER"

- name: Generate portable Dilithium5 certificate
id: cert
shell: bash
run: |
python -c "
import json, os, uuid, hashlib, subprocess
from datetime import datetime, timezone
from ube_foundation import PqcKeyPair

```
def git(cmd):
    try:
        return subprocess.check_output(['git'] + cmd, text=True).strip()
    except Exception:
        return "
```

```
key = PqcKeyPair.generate()
```

```
cert = {
    'spec': 'NTI-Cert/1',
    'issuer': os.environ.get('GITHUB_REPOSITORY', ""),
    'issued_at': datetime.now(timezone.utc).isoformat(),
    'id': str(uuid.uuid4()),
    'repo': os.environ.get('GITHUB_REPOSITORY', ""),
    'commit_sha': os.environ.get('GITHUB_SHA', ""),
    'commit_author': git(['log', '-1', '--format=%ae']),
    'branch': os.environ.get('GITHUB_REF_NAME', ""),
    'run_id': os.environ.get('GITHUB_RUN_ID', ""),
    'score': int(os.environ.get('SCORE', '0')),
    'findings_count': int(os.environ.get('FINDINGS', '0')),
    'tier': os.environ.get('TIER', 'Unranked'),
    'profile': 'NTI-1',
    'language': 'python',
    'signature_alg': 'Dilithium5',
    'issuer_pubkey': key.public_key_hex(),
}
message = json.dumps(cert, sort_keys=True).encode('utf-8')
cert['signature'] = key.sign(message).hex()
cert['cert_hash'] = hashlib.sha256(message).hexdigest()
```

```
with open('nti-certificate.json', 'w') as f:
    json.dump(cert, f, indent=2)
print('Certificate written. Hash: ' + cert['cert_hash'])
"
```

```
env:
    SCORE: ${ steps.scan.outputs.score }
    FINDINGS: ${ steps.scan.outputs.findings }
    TIER: ${ steps.tier.outputs.tier }
    continue-on-error: true
```

```
- name: Enforce minimum score
  if: inputs.fail-below != '0'
  shell: bash
  run: |
```

```

if [ "${{ steps.scan.outputs.score }}" -lt "${{ inputs.fail-below }}" ]; then
  echo "Score below required ${{ inputs.fail-below }}"
  exit 1
fi

```

- name: Submit cert hash to public index (optional)

if: inputs.submit == 'true'

shell: bash

env:

OIDC_URL: \${{ env.ACTIONS_ID_TOKEN_REQUEST_URL }}

OIDC_TOKEN: \${{ env.ACTIONS_ID_TOKEN_REQUEST_TOKEN }}

run: |

set +e

OIDC_JSON=\$(curl -s -H "Authorization: bearer \$OIDC_TOKEN"

"\$OIDC_URL&audience=nti-badge")

ID_TOKEN=\$(echo "\$OIDC_JSON" | python -c "import sys,json;

print(json.load(sys.stdin)['value'])" 2>/dev/null)

if [-z "\$ID_TOKEN"]; then exit 0; fi

CERT=\$(cat nti-certificate.json)

PAYLOAD=\$(python -c "

import json, base64, sys

tok = sys.argv[1]

claims = json.loads(base64.urlsafe_b64decode(tok.split('.')[1] + '=' * (-len(tok.split('.')[1]) % 4)))

cert = json.loads(sys.argv[2])

print(json.dumps({'oidc_claim': claims, 'certificate': cert}))

" "\$ID_TOKEN" "\$CERT")

curl -s -X POST "\${{ inputs.registry-url }}/api/verify" -H "Content-Type: application/json" -d

"\$PAYLOAD" || true

continue-on-error: true

- name: Upload portable certificate

uses: actions/upload-artifact@v4

with:

name: nti-certificate

path: nti-certificate.json

continue-on-error: true

=== FILE 2: entrypoint.sh ===

#!/usr/bin/env bash

set -euo pipefail

echo "NTI Verify action entrypoint"

=== FILE 3: vercel.json ===

{

```

"version": 2,
"builds": [
  {"src": "api/*.py", "use": "@vercel/python"},
  {"src": "public/**", "use": "@vercel/static"}
],
"routes": [
  {"src": "/api/badge", "dest": "api/badge.py"},
  {"src": "/api/verify", "dest": "api/verify.py"},
  {"src": "/api/registry", "dest": "api/registry.py"},
  {"src": "/api/leaderboard", "dest": "api/leaderboard.py"},
  {"src": "/api/certificate", "dest": "api/certificate.py"},
  {"src": "/api/transparency", "dest": "api/transparency.py"},
  {"src": "/api/org", "dest": "api/org.py"},
  {"src": "/api/federation", "dest": "api/federation.py"},
  {"src": "/(.*)", "dest": "public/$1"}
]
}

```

=== FILE 4: requirements.txt ===

```

httpx>=0.27.0
redis>=5.0.0
pyjwt[crypto]>=2.8.0
cryptography>=42.0.0

```

=== FILE 5: LICENSE ===

Apache License 2.0

Copyright 2026 Abisheak

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

=== FILE 6: .gitignore ===

```

__pycache__/
*.pyc

```

```
.venv/  
venv/  
.vercel  
.env  
.env.local  
.DS_Store  
node_modules/  
nti-report.json  
nti-sbom.json
```

Note: nti-certificate.json should be COMMITTED by users to their repo so it becomes part of their permanent audit trail

Chunk 2 of 5. Add the lib/ package.

```
=== FILE 7: lib/__init__.py ===
```

```
"""NTI Badge core library."""
```

```
__version__ = "0.1.0"
```

```
=== FILE 8: lib/spec.py ===
```

```
"""NTI-Cert/1 specification constants."""
```

```
CERT_SPEC = "NTI-Cert/1"
```

```
SUPPORTED_PROFILES = {
```

```
    "NTI-1": {"min_score": 50, "pillars": ["Identity", "Governance", "Consensus", "Audit",  
    "Persistence"]},  
}
```

```
TIERS = {
```

```
    "Platinum": 95,  
    "Gold": 85,  
    "Silver": 70,  
    "Bronze": 50,  
    "Unranked": 0,  
}
```

```
SEVERITY_LEVELS = ["critical", "high", "medium", "low", "info"]
```

```
# OIDC providers the badge accepts (multi-ecosystem)
```

```
SUPPORTED_OIDC_ISSUERS = {
```

```
    "github": "https://token.actions.githubusercontent.com",  
    "gitlab": "https://gitlab.com",
```

```

    "google": "https://accounts.google.com",
    "bitbucket": "https://api.bitbucket.org/2.0/workspaces",
    "azure": "https://login.microsoftonline.com",
}

```

```

def tier_for_score(score: int) -> str:
    if score >= 95: return "Platinum"
    if score >= 85: return "Gold"
    if score >= 70: return "Silver"
    if score >= 50: return "Bronze"
    return "Unranked"

```

=== FILE 9: lib/oidc.py ===

"""Multi-provider OIDC verification.

Supports GitHub Actions, GitLab CI, Google Cloud, Azure DevOps, and custom
OIDC issuers. The badge is ecosystem-agnostic by design — it is a federated
protocol, not a GitHub-only product.

"""

```

import time
from typing import Optional

```

```

import httpx

```

```

JWKS_CACHE = {}
JWKS_TTL = 3600

```

```

PROVIDERS = {
    "github": {
        "config_url":
"https://token.actions.githubusercontent.com/.well-known/openid-configuration",
        "audience": "nti-badge",
        "repo_claim": "repository",
    },
    "gitlab": {
        "config_url": "https://gitlab.com/.well-known/openid-configuration",
        "audience": "nti-badge",
        "repo_claim": "project_path",
    },
    "google": {
        "config_url": "https://accounts.google.com/.well-known/openid-configuration",
        "audience": "nti-badge",
        "repo_claim": "sub",
    },
}

```

```

"azure": {
    "config_url":
"https://login.microsoftonline.com/common/v2.0/.well-known/openid-configuration",
    "audience": "nti-badge",
    "repo_claim": "sub",
},
}

```

```

async def fetch_jwks(provider: str) -> dict:
    entry = JWKS_CACHE.get(provider, {})
    if entry.get("keys") and time.time() - entry.get("fetched_at", 0) < JWKS_TTL:
        return entry["keys"]
    cfg = PROVIDERS.get(provider)
    if not cfg:
        return {}
    async with httpx.AsyncClient(timeout=10) as client:
        r = await client.get(cfg["config_url"])
        r.raise_for_status()
        jwks_uri = r.json().get("jwks_uri")
        r2 = await client.get(jwks_uri)
        r2.raise_for_status()
        JWKS_CACHE[provider] = {"keys": r2.json(), "fetched_at": time.time()}
        return r2.json()

```

```

def detect_provider(payload: dict) -> Optional[str]:
    iss = payload.get("iss", "")
    for name, cfg in PROVIDERS.items():
        if cfg["config_url"].split("/.well-known")[0] in iss:
            return name
    return None

```

```

def verify_oidc_claim(token_payload: dict, expected_repo: Optional[str] = None) ->
Optional[dict]:
    """Validate OIDC claims. Returns normalized identity or None."""
    provider = detect_provider(token_payload)
    if not provider:
        return None

    cfg = PROVIDERS[provider]

    exp = token_payload.get("exp", 0)

```

```

if exp < time.time():
    return None

aud = token_payload.get("aud")
audiences = aud if isinstance(aud, list) else [aud] if aud else []
if cfg["audience"] not in audiences:
    return None

repo = token_payload.get(cfg["repo_claim"])
if not repo:
    return None

# Normalize: strip URL prefixes for gitlab/google/azure
if provider == "gitlab" and isinstance(repo, str):
    repo = repo.rstrip("/")
if expected_repo and repo != expected_repo:
    return None

return {
    "provider": provider,
    "repo_full": repo,
    "subject": token_payload.get("sub", ""),
    "issuer": token_payload.get("iss", ""),
    "raw_claims": {
        "sha": token_payload.get("sha") or token_payload.get("commit_sha", ""),
        "ref": token_payload.get("ref", ""),
        "actor": token_payload.get("actor") or token_payload.get("user_login", ""),
    },
}

```

=== FILE 10: lib/store.py ===

"""Federated-friendly storage.

Design principle: NTI stores ONLY what is needed to index certificates — never the full certificate, never the user's data. This keeps the registry neutral and lets anyone run a mirror node.

"""

```

import os
import json
from typing import Optional

```

```

try:
    import redis.asyncio as aioredis
    KV_URL = os.environ.get("KV_URL") or os.environ.get("REDIS_URL")

```

```
    _redis = aioredis.from_url(KV_URL, decode_responses=True) if KV_URL else None
except Exception:
    _redis = None
```

```
async def index_certificate(repo_full: str, entry: dict) -> None:
    """Store only the index entry (hash + score + timestamp)."""
    if not _redis:
        return
    payload = json.dumps(entry)
    await _redis.hset("nti:index", repo_full, payload)
    await _redis.zadd("nti:leaderboard", {repo_full: entry.get("score", 0)})
    # Append-only history (for drift detection + transparency pages)
    await _redis.rpush(f"nti:history:{repo_full}", payload)
    # Cap history at 500 entries per repo to bound storage
    await _redis.ltrim(f"nti:history:{repo_full}", -500, -1)
```

```
async def get_latest(repo_full: str) -> Optional[dict]:
    if not _redis:
        return None
    raw = await _redis.hget("nti:index", repo_full)
    return json.loads(raw) if raw else None
```

```
async def get_history(repo_full: str, limit: int = 50) -> list:
    if not _redis:
        return []
    raw_entries = await _redis.lrange(f"nti:history:{repo_full}", -limit, -1)
    return [json.loads(e) for e in raw_entries]
```

```
async def list_all() -> list:
    if not _redis:
        return []
    entries = await _redis.hgetall("nti:index")
    return [{"repo": k, **json.loads(v)} for k, v in entries.items()]
```

```
async def top_scores(limit: int = 50) -> list:
    if not _redis:
        return []
    results = await _redis.zrevrange("nti:leaderboard", 0, limit - 1, withscores=True)
    out = []
```

```

for repo, score in results:
    entry = await get_latest(repo)
    if entry:
        out.append({"repo": repo, "score": int(score), "timestamp": entry.get("timestamp", "")})
return out

```

```

async def list_by_org(org: str) -> list:
    if not _redis:
        return []
    all_entries = await list_all()
    return [e for e in all_entries if e["repo"].startswith(f"{org}/")]

```

=== FILE 11: lib/crypto.py ===

"""Certificate signature verification helpers.

Verifies Dilithium5 signatures on NTI-Cert/1 payloads. Supports offline verification — no network required. This is what makes the certs portable.

"""

```

import json
import hashlib
from typing import Optional

```

```

def compute_cert_hash(payload: dict) -> str:
    """Deterministic hash of the cert payload without the signature field."""
    clean = {k: v for k, v in payload.items() if k not in ("signature", "cert_hash")}
    message = json.dumps(clean, sort_keys=True).encode("utf-8")
    return hashlib.sha256(message).hexdigest()

```

```

def verify_dilithium_signature(payload: dict) -> bool:
    """Verify the signature against the embedded public key.

```

Uses ube-foundation if available. Falls back to hash-only check.

"""

```

    sig_hex = payload.get("signature")
    pk_hex = payload.get("issuer_pubkey")
    if not sig_hex or not pk_hex:
        return False
    try:
        from ube_foundation import PqcKeyPair
        clean = {k: v for k, v in payload.items() if k not in ("signature", "cert_hash")}
        message = json.dumps(clean, sort_keys=True).encode("utf-8")

```

```

        key = PqcKeyPair.from_public_key_hex(pk_hex) if hasattr(PqcKeyPair,
"from_public_key_hex") else None
        if key is None:
            # Fallback: recompute hash integrity
            return compute_cert_hash(payload) == payload.get("cert_hash")
        return key.verify(message, sig_hex)
    except Exception:
        return False

```

```

def verify_cert_integrity(payload: dict) -> bool:
    """Verify cert hash matches payload (integrity check)."""
    return compute_cert_hash(payload) == payload.get("cert_hash")

```

=== FILE 12: lib/certificate.py ===

"""NTI-Cert/1 certificate parsing and validation."""

from typing import Optional

```

from lib.spec import CERT_SPEC, SUPPORTED_PROFILES
from lib.crypto import verify_cert_integrity, verify_dilithium_signature

```

```

REQUIRED_FIELDS = [
    "spec", "issuer", "issued_at", "id", "repo", "commit_sha",
    "score", "profile", "signature_alg", "issuer_pubkey",
    "signature", "cert_hash",
]

```

```

def parse_certificate(payload: dict) -> dict:
    """Validate structure and return normalized cert or raise ValueError."""
    missing = [f for f in REQUIRED_FIELDS if f not in payload]
    if missing:
        raise ValueError(f"Missing required fields: {missing}")

    if payload["spec"] != CERT_SPEC:
        raise ValueError(f"Unsupported spec: {payload['spec']}")

    if payload["profile"] not in SUPPORTED_PROFILES:
        raise ValueError(f"Unsupported profile: {payload['profile']}")

    if not isinstance(payload["score"], int) or not (0 <= payload["score"] <= 100):
        raise ValueError("Score must be integer 0-100")

```

```

if not verify_cert_integrity(payload):
    raise ValueError("Certificate hash mismatch — payload has been tampered with")

return payload

```

```

def is_valid_signature(payload: dict) -> bool:
    return verify_dilithium_signature(payload)

```

```

def to_index_entry(payload: dict) -> dict:
    """Extract only the fields needed for the public registry index."""
    return {
        "repo": payload["repo"],
        "score": payload["score"],
        "cert_hash": payload["cert_hash"],
        "commit_sha": payload["commit_sha"],
        "timestamp": payload["issued_at"],
        "issuer_pubkey": payload["issuer_pubkey"],
        "signature": payload["signature"],
        "tier": payload.get("tier", "Unranked"),
        "profile": payload["profile"],
    }

```

=== FILE 13: lib/tiers.py ===

```

"""Compliance tier logic."""
from lib.spec import tier_for_score, TIERS

```

```

TIER_COLORS = {
    "Platinum": "#e5e4e2",
    "Gold": "#ffd700",
    "Silver": "#c0c0c0",
    "Bronze": "#cd7f32",
    "Unranked": "#6b7280",
}

```

```

TIER_BADGES = {
    "Platinum": "🏆",
    "Gold": "🥇",
    "Silver": "🥈",
    "Bronze": "🥉",
    "Unranked": "🏠",
}

```

```
def assign_tier(score: int) -> str:
    return tier_for_score(score)
```

```
def tier_metadata(tier: str) -> dict:
    return {
        "name": tier,
        "color": TIER_COLORS.get(tier, "#888"),
        "emoji": TIER_BADGES.get(tier, ""),
        "threshold": TIERS.get(tier, 0),
    }
```

=== FILE 14: lib/drift.py ===

"""Score drift detection.

Compares the new cert's score to the previous cert for the same repo.

Triggers alerts on regressions.

"""

```
def compute_drift(previous: dict, current: dict) -> dict:
    prev_score = previous.get("score", 0) if previous else None
    curr_score = current.get("score", 0)
    if prev_score is None:
        return {"direction": "first_scan", "delta": 0, "previous": None, "current": curr_score}
    delta = curr_score - prev_score
    if delta > 0:
        direction = "improved"
    elif delta < 0:
        direction = "regressed"
    else:
        direction = "stable"
    return {
        "direction": direction,
        "delta": delta,
        "previous": prev_score,
        "current": curr_score,
    }
```

```
def is_significant_regression(drift: dict, threshold: int = 10) -> bool:
    return drift["direction"] == "regressed" and abs(drift["delta"]) >= threshold
```

Chunk 3 of 5. Add the api/ serverless endpoints.

=== FILE 15: api/badge.py ===

"""GET /api/badge?repo=owner/repo — serves verified SVG badge.

Renders a tiered badge based on the latest indexed cert for the repo.

If no cert is indexed, returns a grey "not verified" badge.

"""

from http.server import BaseHTTPRequestHandler

from urllib.parse import urlparse, parse_qs

import asyncio

from lib.store import get_latest

from lib.tiers import tier_metadata

def _badge_svg(label: str, value: str, color: str, tier_emoji: str = "") -> str:

label_w = 120

value_w = 140

total = label_w + value_w

return f"<svg xmlns='http://www.w3.org/2000/svg' width='{total}' height='22' role='img' aria-label='{label}: {value}'>

<title>{label}: {value}</title>

<linearGradient id='s' x2='0' y2='100%'>

<stop offset='0' stop-color='#bbb' stop-opacity='.1'>

<stop offset='1' stop-opacity='.1'>

</linearGradient>

<clipPath id='r'><rect width='{total}' height='22' rx='4' fill='#fff'></clipPath>

<g clip-path='url(#r)'>

<rect width='{label_w}' height='22' fill='#0a0a0a'>

<rect x='{label_w}' width='{value_w}' height='22' fill='{color}'>

<rect width='{total}' height='22' fill='url(#s)'>

</g>

<g fill='#fff' text-anchor='middle' font-family='Verdana, Geneva, sans-serif' font-size='11'>

<text x='{label_w // 2}' y='15'>NTI-1 {tier_emoji}</text>

<text x='{label_w + value_w // 2}' y='15'>{value}</text>

</g>

</svg>"""

class handler(BaseHTTPRequestHandler):

def do_GET(self):

params = parse_qs(urlparse(self.path).query)

```

repo = params.get("repo", [""])[0]

if not repo or "/" not in repo:
    self._send(_badge_svg("NTI-1", "not verified", "#555"))
    return

try:
    entry = asyncio.run(get_latest(repo))
except Exception:
    entry = None

if not entry:
    self._send(_badge_svg("NTI-1", "not verified", "#555"))
    return

score = entry.get("score", 0)
tier = entry.get("tier", "Unranked")
meta = tier_metadata(tier)

self._send(_badge_svg("NTI-1", f"{score}/100 {tier}", meta["color"], meta["emoji"]))

def _send(self, svg):
    self.send_response(200)
    self.send_header("Content-Type", "image/svg+xml; charset=utf-8")
    self.send_header("Cache-Control", "public, max-age=300")
    self.end_headers()
    self.wfile.write(svg.encode("utf-8"))

```

=== FILE 16: api/verify.py ===

"""POST /api/verify — receives signed certificate submissions.

Validates:

1. OIDC token (proves the request came from the claimed repo)
2. Certificate integrity (hash matches payload)
3. Certificate signature (Dilithium5)
4. Repo claim in OIDC matches repo in certificate

Indexes ONLY the hash + score. Full cert stays in the user's repo.

"""

```

from http.server import BaseHTTPRequestHandler
import json
import asyncio

```

```

from lib.oidc import verify_oidc_claim

```

```
from lib.store import index_certificate, get_latest
from lib.certificate import parse_certificate, is_valid_signature, to_index_entry
from lib.drift import compute_drift
```

```
class handler(BaseHTTPRequestHandler):
    def do_POST(self):
        try:
            length = int(self.headers.get("content-length", 0))
            body = json.loads(self.rfile.read(length).decode("utf-8"))
        except Exception:
            self._respond(400, {"error": "invalid json"})
            return

        oidc_claim = body.get("oidc_claim", {})
        cert_raw = body.get("certificate", {})

        # Step 1: Validate OIDC token
        identity = verify_oidc_claim(oidc_claim)
        if not identity:
            self._respond(401, {"error": "invalid or expired oidc token"})
            return

        # Step 2: Validate certificate structure
        try:
            cert = parse_certificate(cert_raw)
        except ValueError as e:
            self._respond(400, {"error": f"invalid certificate: {e}"})
            return

        # Step 3: Cross-check repo identity
        if identity["repo_full"] != cert["repo"]:
            self._respond(403, {"error": "oidc repo does not match certificate repo"})
            return

        # Step 4: Verify Dilithium5 signature (best-effort)
        sig_valid = is_valid_signature(cert)

        # Step 5: Compute drift vs previous
        try:
            previous = asyncio.run(get_latest(cert["repo"]))
        except Exception:
            previous = None
        drift = compute_drift(previous, cert)
```

```

# Step 6: Index (hash + score only)
index_entry = to_index_entry(cert)
index_entry["signature_verified"] = sig_valid
index_entry["drift"] = drift

try:
    asyncio.run(index_certificate(cert["repo"], index_entry))
except Exception as e:
    self._respond(500, {"error": f"index failed: {e}"})
    return

self._respond(200, {
    "status": "indexed",
    "repo": cert["repo"],
    "score": cert["score"],
    "tier": cert.get("tier", "Unranked"),
    "signature_verified": sig_valid,
    "drift": drift,
    "note": "Only cert_hash and score are indexed. Full certificate remains in your repository.",
})

def _respond(self, code, payload):
    self.send_response(code)
    self.send_header("Content-Type", "application/json")
    self.send_header("Access-Control-Allow-Origin", "**")
    self.end_headers()
    self.wfile.write(json.dumps(payload).encode("utf-8"))

=== FILE 17: api/registry.py ===
"""GET /api/registry — JSON list of all indexed certs."""
from http.server import BaseHTTPRequestHandler
import json
import asyncio

from lib.store import list_all

class handler(BaseHTTPRequestHandler):
    def do_GET(self):
        try:
            entries = asyncio.run(list_all())
        except Exception:

```

```

        entries = []
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Access-Control-Allow-Origin", "")
        self.end_headers()
        self.wfile.write(json.dumps({
            "spec": "NTI-Cert/1",
            "count": len(entries),
            "note": "This is a federated index. Full certificates live in each repo. Anyone can mirror
this endpoint.",
            "repos": entries,
        }, indent=2).encode("utf-8"))

```

=== FILE 18: api/leaderboard.py ===

```

"""GET /api/leaderboard — top 50 repos by score."""
from http.server import BaseHTTPRequestHandler
import json
import asyncio

```

```

from lib.store import top_scores

```

```

class handler(BaseHTTPRequestHandler):
    def do_GET(self):
        try:
            entries = asyncio.run(top_scores(50))
        except Exception:
            entries = []
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Access-Control-Allow-Origin", "")
        self.end_headers()
        self.wfile.write(json.dumps({"leaderboard": entries}, indent=2).encode("utf-8"))

```

=== FILE 19: api/certificate.py ===

```

"""GET /api/certificate?repo=owner/repo — returns the latest indexed cert metadata.

```

Note: this returns the INDEX entry (hash + score + metadata), not the full certificate. The full certificate lives in the user's repository. To verify a full certificate, use the offline verification tool in nti-scanner.

```

"""

```

```

from http.server import BaseHTTPRequestHandler
from urllib.parse import urlparse, parse_qs
import json

```

```

import asyncio

from lib.store import get_latest, get_history

class handler(BaseHTTPRequestHandler):
    def do_GET(self):
        params = parse_qs(urlparse(self.path).query)
        repo = params.get("repo", [""])[0]
        if not repo or "/" not in repo:
            self._respond(400, {"error": "repo parameter required (owner/repo)"})
            return

        try:
            latest = asyncio.run(get_latest(repo))
            history = asyncio.run(get_history(repo, 50))
        except Exception:
            latest = None
            history = []

        self._respond(200, {
            "repo": repo,
            "latest": latest,
            "history_count": len(history),
            "history": history,
            "how_to_verify_offline": "pip install nti-scanner && nti-scanner verify ./nti-certificate.json",
        })

    def _respond(self, code, payload):
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Access-Control-Allow-Origin", "")
        self.end_headers()
        self.wfile.write(json.dumps(payload, indent=2).encode("utf-8"))

```

```

=== FILE 20: api/transparency.py ===
"""GET /api/transparency — public transparency log.

```

Every indexed cert hash with its timestamp, forming an append-only public log.
 Anyone can rebuild and audit this. ZERO-knowledge: only hashes are visible.
 """

```

from http.server import BaseHTTPRequestHandler
import json
import asyncio

```

```
from lib.store import list_all
```

```
class handler(BaseHTTPRequestHandler):
    def do_GET(self):
        try:
            entries = asyncio.run(list_all())
        except Exception:
            entries = []
        log = sorted(
            [{"repo": e["repo"], "cert_hash": e.get("cert_hash", ""), "timestamp": e.get("timestamp",
            ""), "score": e.get("score")}] for e in entries],
            key=lambda x: x.get("timestamp", ""),
        )
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Access-Control-Allow-Origin", "")
        self.end_headers()
        self.wfile.write(json.dumps({
            "log_type": "nti-transparency/1",
            "entry_count": len(log),
            "entries": log,
        }, indent=2).encode("utf-8"))
```

```
=== FILE 21: api/org.py ===
```

```
"""GET /api/org?name=<org> — aggregate all repos under an org or user."""
```

```
from http.server import BaseHTTPRequestHandler
```

```
from urllib.parse import urlparse, parse_qs
```

```
import json
```

```
import asyncio
```

```
from lib.store import list_by_org
```

```
from lib.tiers import assign_tier
```

```
class handler(BaseHTTPRequestHandler):
    def do_GET(self):
        params = parse_qs(urlparse(self.path).query)
        org = params.get("name", [""])[0]
        if not org:
            self._respond(400, {"error": "name parameter required"})
            return
```

```

try:
    repos = asyncio.run(list_by_org(org))
except Exception:
    repos = []

if not repos:
    self._respond(200, {"org": org, "repo_count": 0, "repos": [], "aggregate_score": 0})
    return

scores = [r["score"] for r in repos if isinstance(r.get("score"), int)]
aggregate = int(sum(scores) / len(scores)) if scores else 0
worst = min(scores) if scores else 0
best = max(scores) if scores else 0

self._respond(200, {
    "org": org,
    "repo_count": len(repos),
    "aggregate_score": aggregate,
    "worst_score": worst,
    "best_score": best,
    "tier": assign_tier(aggregate),
    "repos": sorted(repos, key=lambda r: r.get("score", 0), reverse=True),
})

def _respond(self, code, payload):
    self.send_response(code)
    self.send_header("Content-Type", "application/json")
    self.send_header("Access-Control-Allow-Origin", "**")
    self.end_headers()
    self.wfile.write(json.dumps(payload, indent=2).encode("utf-8"))

```

=== FILE 22: api/federation.py ===

"""GET /api/federation — federation protocol manifest.

This endpoint makes NTI a protocol, not a product. Any organization can read this manifest and stand up their own mirror. That is what makes NTI neutral and un-killable.

"""

```

from http.server import BaseHTTPRequestHandler
import json

```

```

MANIFEST = {
    "protocol": "NTI-Cert/1",

```

```

"federation_version": "1.0",
"description": (
    "NTI is a federated certificate protocol. This registry is one node. "
    "Any party may run a mirror or their own private federation. "
    "Certificates are portable and verifiable offline."
),
"endpoints": {
    "verify": "POST /api/verify",
    "badge": "GET /api/badge?repo=owner/repo",
    "certificate": "GET /api/certificate?repo=owner/repo",
    "registry": "GET /api/registry",
    "leaderboard": "GET /api/leaderboard",
    "transparency": "GET /api/transparency",
    "org": "GET /api/org?name=<org>",
    "federation_manifest": "GET /api/federation",
},
"mirror_howto": {
    "step_1": "Clone github.com/abisheakp197/nti-badge",
    "step_2": "Deploy to Vercel (or any Python serverless platform)",
    "step_3": "Attach a KV/Redis store",
    "step_4": "Announce your mirror. Clients can use it via the registry-url input.",
},
"cert_format": {
    "spec": "NTI-Cert/1",
    "signature_alg": "Dilithium5",
    "fields_required": [
        "spec", "issuer", "issued_at", "id", "repo", "commit_sha",
        "score", "profile", "signature_alg", "issuer_pubkey",
        "signature", "cert_hash",
    ],
},
"verification_offline": "nti-scanner verify ./nti-certificate.json",
"license": "Apache-2.0",
"neutrality": (
    "No central authority. No data custody. Certificates live in the user's "
    "repository. This registry indexes only {repo, score, cert_hash, timestamp}."
),
}

```

```

class handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "application/json")

```

```

self.send_header("Access-Control-Allow-Origin", "")
self.end_headers()
self.wfile.write(json.dumps(MANIFEST, indent=2).encode("utf-8"))

```

Chunk 4 of 5. Add the public pages, documentation, and README.

=== FILE 23: public/index.html ===

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>NTI-1 Verified Badge — Federated AI Agent Compliance</title>
<style>
:root{--bg:#0a0a0a;--fg:#eee;--muted:#888;--accent:#4ade80;--card:#111;--border:#222}
*{box-sizing:border-box}
body{font-family:system-ui,-apple-system,sans-serif;background:var(--bg);color:var(--fg);padding:
:2rem;max-width:920px;margin:auto;line-height:1.65}
h1,h2,h3{color:#fff;line-height:1.25}
h1{font-size:2.2rem;margin-bottom:.5rem}
code{background:var(--card);padding:.15rem
.45rem;border-radius:5px;color:var(--accent);font-size:.92em;font-family:ui-monospace,SFMono-
Regular,Menlo,monospace}
pre{background:var(--card);padding:1rem
1.2rem;border-radius:10px;overflow-x:auto;color:var(--accent);border:1px solid
var(--border);font-size:.88em;line-height:1.5}
a{color:#22d3ee;text-decoration:none}
a:hover{text-decoration:underline}
.muted{color:var(--muted)}
.card{background:var(--card);border:1px solid
var(--border);border-radius:12px;padding:1.5rem;margin:1.5rem 0}
.badge-row{display:flex;gap:1rem;flex-wrap:wrap;margin:1rem 0}
hr{border:0;border-top:1px solid var(--border);margin:2.5rem 0}
.pill{display:inline-block;background:#1a1a1a;border:1px solid var(--border);padding:.2rem
.7rem;border-radius:999px;font-size:.82rem;margin-right:.4rem;color:var(--accent)}
</style>
</head>
<body>

<h1>NTI-1 Verified Badge</h1>
<p class="muted">Federated, post-quantum signed, zero-lock-in compliance certificates for AI
agent projects.</p>

<div class="badge-row">

```

```
<span class="pill">Dilithium5 signed</span>
<span class="pill">Multi-OIDC</span>
<span class="pill">Offline verifiable</span>
<span class="pill">Federated</span>
<span class="pill">Zero data lock-in</span>
</div>
```

```
<div class="card">
<h3>Live badge</h3>
<div class="badge-row">
  
  
</div>
</div>
```

```
<hr>
```

<h2>The problem this solves</h2>

<p>Every enterprise deploying AI agents has the same question: "Is this agent actually secure, and can I prove it to an auditor?" Today, the answer is a marketing PDF. It cannot be verified. It cannot be trusted.</p>

<p>NTI gives every AI agent repo a portable, cryptographically signed compliance certificate that any third party can verify offline in seconds. No server. No subscription. No lock-in.</p>

```
<hr>
```

<h2>Why this is different</h2>

||
||
||

Install in 2 steps

1. Add this to .github/workflows/nti.yml

```
name: NTI Verify
on: [push, pull_request]
permissions:
  id-token: write
  contents: read
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: abisheakp197/nti-badge@v1
      - name: Commit certificate
        run: |
          git config user.name "nti-bot"
          git config user.email "bot@nti.local"
          git add nti-certificate.json
          git commit -m "chore: NTI certificate [skip ci]" || true
          git push || true
```

2. Add the badge to your README

```
<![NTI-1
Verified](https://nti-badge.vercel.app/api/badge?repo=your-org/your-repo)](https://nti-badge.vercel.app/repo.html?repo=your-org/your-repo)>
```

Verify any certificate offline

```
pip install nti-scanner
nti-scanner verify ./nti-certificate.json
```

No network. No NTI servers. Just math.

Federation

NTI is a protocol, not a product. Any company, university, or government can run their own mirror node. See the [federation manifest](/api/federation) for the full spec.

```

<h2>Endpoints</h2>
<ul>
<li><a href="/api/registry">/api/registry</a> — index of all verified repos</li>
<li><a href="/api/leaderboard">/api/leaderboard</a> — top 50 by score</li>
<li><a href="/api/transparency">/api/transparency</a> — append-only public log</li>
<li><a href="/api/federation">/api/federation</a> — federation manifest</li>
<li><a href="/api/certificate?repo=abisheakp197/nti-scanner">/api/certificate</a> — index entry
for a repo</li>
<li><a href="/api/org?name=abisheakp197">/api/org</a> — org-level aggregate score</li>
</ul>

```

```

<h2>Links</h2>
<ul>
<li><a href="https://abisheakp197.github.io/nti-spec/">NTI-1 Specification</a></li>
<li><a href="https://pypi.org/project/nti-scanner/">nti-scanner (PyPI)</a></li>
<li><a href="https://pypi.org/project/ube-foundation/">ube-foundation (PyPI)</a></li>
<li><a href="https://github.com/abisheakp197/nti-badge">Source code</a></li>
</ul>

```

```

</body>
</html>

```

=== FILE 24: public/repo.html ===

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>NTI-1 Repo Transparency</title>
<style>
body{font-family:system-ui;background:#0a0a0a;color:#eee;padding:2rem;max-width:900px;margin:auto;line-height:1.6}
h1,h2{color:#fff}
code,pre{background:#111;padding:.2rem .5rem;border-radius:5px;color:#4ade80;font-family:ui-monospace,monospace}
pre{padding:1rem;overflow-x:auto;border:1px solid #222}
table{width:100%;border-collapse:collapse;margin:1rem 0}
th,td{text-align:left;padding:.6rem;border-bottom:1px solid #222}
th{color:#888;font-weight:500;font-size:.85rem}
a{color:#22d3ee}
#output{margin-top:1rem}
.muted{color:#888}
</style>
</head>

```

```

<body>
<h1>Repo Transparency Page</h1>
<p class="muted">Look up the NTI-1 verification history of any repo.</p>

<form id="lookup" onsubmit="return false">
<input id="repo" placeholder="owner/repo"
style="padding:.6rem;background:#111;color:#eee;border:1px solid
#333;border-radius:6px;width:60%">
<button style="padding:.6rem
1rem;background:#4ade80;color:#000;border:0;border-radius:6px;font-weight:600;cursor:pointer
" onclick="loadRepo()">Look up</button>
</form>

<div id="output"></div>

<script>
function esc(s){return String(s ?? "").replace(/[\<>"]/g, c =>
({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;','\':'&#39;'}[c]));}
async function loadRepo(){
  const repo = document.getElementById('repo').value.trim();
  const out = document.getElementById('output');
  if (!repo.includes('/')) { out.innerHTML = '<p style="color:#ef4444">Use owner/repo format</p>';
return; }
  out.innerHTML = '<p class="muted">Loading...</p>';
  try {
    const r = await fetch('/api/certificate?repo=' + encodeURIComponent(repo));
    const data = await r.json();
    if (!data.latest) { out.innerHTML = '<p>No NTI-1 certificate indexed for this repo yet.</p>';
return; }
    let html = '<h2>Latest verified certificate</h2>';
    html += '<table><tr><th>Field</th><th>Value</th></tr>';
    const fields = ['repo','score','tier','commit_sha','timestamp','cert_hash','signature_verified'];
    for (const f of fields) {
      if (data.latest[f] !== undefined) html += '<tr><td>' + esc(f) + '</td><td><code>' +
esc(data.latest[f]) + '</code></td></tr>';
    }
    html += '</table>';
    html += '<h2>History (' + data.history_count + ' entries)</h2>';
    html += '<table><tr><th>Timestamp</th><th>Score</th><th>Tier</th><th>Commit</th></tr>';
    for (const h of (data.history||[]).slice().reverse().slice(0,25)) {
      html += '<tr><td>' + esc(h.timestamp) + '</td><td>' + esc(h.score) + '</td><td>' +
esc(h.tier||'') + '</td><td><code>' + esc((h.commit_sha||'').slice(0,8)) + '</code></td></tr>';
    }
    html += '</table>';
  }
}

```

```

    html += '<h2>Verify offline</h2><pre>pip install nti-scanner\nnti-scanner verify
./nti-certificate.json</pre>';
    out.innerHTML = html;
  } catch (e) { out.innerHTML = '<p style="color:#ef4444">Lookup failed.</p>'; }
}
</script>
</body>
</html>

```

=== FILE 25: docs/NTI-CERT-SPEC.md ===

NTI-Cert/1 Specification

Version: 1.0

Status: Stable

License: Apache 2.0

Purpose

NTI-Cert/1 is a portable, post-quantum-signed compliance certificate format for autonomous AI agent repositories. Any repo can produce an NTI-Cert/1 certificate. Any party can verify it offline. No central authority is required.

Design Principles

1. **Portable** — the certificate travels with the code. It is never held hostage.
2. **Verifiable offline** — verification requires only the cert and the spec.
3. **Post-quantum** — Dilithium5 signatures (NIST Level 5).
4. **Commit-bound** — every cert references a specific commit SHA.
5. **Append-only** — certificates accumulate. History is the audit trail.
6. **Federated** — any party may index certs. No single registry is authoritative.
7. **Zero data custody** — registries index hashes only, never full payloads.

Required Fields

Field	Type	Description
spec	string	Always "NTI-Cert/1"
issuer	string	The repo that issued the cert
issued_at	string	ISO 8601 UTC timestamp
id	string	UUIDv4, unique per cert
repo	string	"owner/repo" format
commit_sha	string	Git commit the cert refers to
score	integer	0–100
profile	string	"NTI-1"

signature_alg	string	Always `"Dilithium5"`
issuer_pubkey	string	Hex-encoded Dilithium5 public key
signature	string	Hex-encoded Dilithium5 signature
cert_hash	string	SHA-256 of payload (excluding `signature` and `cert\_hash`)

Optional Fields

Field	Type	Description
tier	string	`Platinum` / `Gold` / `Silver` / `Bronze` / `Unranked`
commit_author	string	Email of the commit author
branch	string	Git branch
run_id	string	CI run identifier
findings_count	integer	Total findings from nti-scanner
language	string	Primary language
issuer_contact	string	Contact URL for the issuer
sbom	object	Optional SBOM attestation

Signature Algorithm

- **Algorithm:** CRYSTALS-Dilithium5 (NIST PQC Level 5)
- **Message:** JSON-serialized payload, `sort\_keys=True`, without `signature` and `cert\_hash`
- **Encoding:** Hex string

Verification Algorithm

1. Remove `signature` and `cert\_hash` from the payload
2. Serialize the remaining fields as JSON with `sort\_keys=True`
3. Compute SHA-256 → must equal `cert\_hash`
4. Use `issuer\_pubkey` to verify `signature` against the message

Federation Protocol

A federation node MUST expose at minimum:

- `POST /api/verify` — accepts signed cert submissions
- `GET /api/certificate?repo=...` — returns index entry for a repo
- `GET /api/federation` — manifest

A federation node MUST NOT:

- Store full certificates (only hashes + scores)
- Require registration to verify certificates
- Have any exclusive control over the cert format

Reference Implementation

- Badge server: ``github.com/abisheakp197/nti-badge``
- Verifier CLI: ``pip install nti-scanner && nti-scanner verify ./nti-certificate.json``
- Core SDK: ``pip install ube-foundation``

Versioning

NTI-Cert/1 is frozen. Future versions will be NTI-Cert/2, etc. Backwards compatibility is required for index parsing.

=== FILE 26: README.md ===

NTI-1 Verified Badge

****A federated, post-quantum-signed compliance certificate system for AI agent repositories.****

[![PyPI](https://img.shields.io/badge/pypi-ube--foundation-blue)](https://pypi.org/project/ube-foundation/)

[![License: Apache 2.0](https://img.shields.io/badge/License-Apache%202.0-blue.svg)](https://opensource.org/licenses/Apache-2.0)

What This Is

The NTI-1 Badge is not a badge. It is a protocol. It gives every AI agent repository a ****portable, cryptographically signed compliance certificate**** that any third party can verify offline. No subscription. No data lock-in. No central authority.

What Makes It Different

Traditional badge	NTI-1 Verified Badge
---	---
Self-reported score	OIDC + Dilithium5 signed
Vendor holds your data	Your repo holds the cert
GitHub only	GitHub, GitLab, Azure, Google, custom
Cannot verify offline	Verify with one command, no network
Centralized	Federated — anyone can mirror
Marketing PDF	Cryptographic proof

Your Certificate, Your Data

Every scan produces a ``nti-certificate.json`` file that lives in ****your**** repository — not on our servers. It is Dilithium5-signed, tied to a specific commit, and verifiable offline with a single command. This is why NTI is a neutral, federated standard: you own your compliance history

forever. We only index a cryptographic hash so the world can see it. If NTI disappears tomorrow, your proof survives.

Install in 2 Steps

1. Add the workflow

```
name: NTI Verify
on: [push, pull_request]
permissions:
  id-token: write
  contents: read
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: abisheakp197/nti-badge@v1
      - name: Commit certificate
        run: |
          git config user.name "nti-bot"
          git config user.email "bot@nti.local"
          git add nti-certificate.json
          git commit -m "chore: NTI certificate [skip ci]" || true
          git push || true
```

2. Add the badge to your README

```
[[NTI-1
Verified](https://nti-badge.vercel.app/api/badge?repo=your-org/your-repo)](https://nti-badge.vercel.app/repo.html?repo=your-org/your-repo)
```

Verify Offline

```
pip install nti-scanner
nti-scanner verify ./nti-certificate.json
```

No network. No NTI servers. Just math.

Federation

Any organization can run a mirror node. Read `/api/federation` for the manifest. Deploy this repo to Vercel, attach a KV store, announce your mirror.

Endpoints

- `POST /api/verify` — submit cert for indexing
- `GET /api/badge?repo=owner/repo` — SVG badge
- `GET /api/certificate?repo=owner/repo` — index entry + history
- `GET /api/registry` — full index
- `GET /api/leaderboard` — top 50
- `GET /api/transparency` — append-only public log
- `GET /api/org?name=<org>` — org aggregate
- `GET /api/federation` — federation manifest

The Certificate Spec

See [docs/NTI-CERT-SPEC.md](./docs/NTI-CERT-SPEC.md).

License

Apache License 2.0. See LICENSE.

Links

- NTI-1 Spec: <https://abisheakp197.github.io/nti-spec/>
- nti-scanner: <https://pypi.org/project/nti-scanner/>
- ube-foundation: <https://pypi.org/project/ube-foundation/>

Chunk 5 of 5 (final). After adding the last file below, execute the deployment and PR steps.

=== FILE 27: .github/workflows/validate.yml ===

name: Validate Badge System

on:

push:

branches: [main]

pull_request:

branches: [main]

jobs:

validate:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-python@v5

with:

python-version: "3.11"

```

- name: Verify Python syntax (all files)
  run: |
    python -m compileall api lib
    echo "All Python files compile successfully"
- name: Verify required files present
  run: |
    test -f action.yml
    test -f vercel.json
    test -f requirements.txt
    test -f README.md
    test -f LICENSE
    test -f docs/NTI-CERT-SPEC.md
    test -f public/index.html
    test -f public/repo.html
    test -f api/badge.py
    test -f api/verify.py
    test -f api/registry.py
    test -f api/leaderboard.py
    test -f api/certificate.py
    test -f api/transparency.py
    test -f api/org.py
    test -f api/federation.py
    test -f lib/oidc.py
    test -f lib/store.py
    test -f lib/crypto.py
    test -f lib/certificate.py
    test -f lib/drift.py
    test -f lib/tiers.py
    test -f lib/spec.py
    echo "All required files present"
- name: Validate vercel.json is valid JSON
  run: python -c "import json; json.load(open('vercel.json'))"
- name: Validate action.yml structure
  run: |
    python -c "
    import sys
    try:
        import yaml
    except ImportError:
        print('PyYAML not installed, skipping deep validation')
        sys.exit(0)
    with open('action.yml') as f:
        data = yaml.safe_load(f)
    assert data['name'] == 'NTI Verify'

```

```

    assert data['runs']['using'] == 'composite'
    print('action.yml valid')
    "
    continue-on-error: true

```

```

=====
=====
FINAL EXECUTION STEPS (after creating the last file above)
=====
=====

```

1. Verify the file tree matches this exact structure:

```

nti-badge/
├── action.yml
├── entrypoint.sh
├── api/          (8 files)
├── lib/          (8 files)
├── public/       (2 files)
├── docs/         (1 file)
├── .github/workflows/ (1 file)
├── vercel.json
├── requirements.txt
├── README.md
├── LICENSE
└── .gitignore

```

2. Push ALL files to the main branch with commit message:
"feat: initial NTI-1 verified badge system — federated, Dilithium5-signed, zero-lock-in"
3. Create a git tag `v1` on the main branch.
4. Create a branch `feat/initial-badge` and open a pull request against `main` with title:
"feat: initial NTI-1 verified badge system (federated, OIDC-verified, unspoofable)"

PR body:

```

"""

```

Summary

This PR introduces the NTI-1 Verified Badge — a federated, post-quantum-signed compliance certificate system for AI agent repositories.

Key Differentiators

- OIDC + Dilithium5 signature verification (unspoofable)
- Portable certificates (user owns them, never NTI)
- Federated protocol (anyone can run a mirror)

- Multi-OIDC provider support (GitHub, GitLab, Azure, Google, custom)
- Offline verification via `nti-scanner verify`
- Zero data custody (index holds only hashes)
- Compliance tiers: Bronze / Silver / Gold / Platinum
- Public transparency log
- Enterprise org aggregation

Files Added

- Reusable GitHub Action (action.yml at root)
- 8 Vercel serverless endpoints
- 8 shared library modules
- Landing page + repo transparency page
- NTI-Cert/1 specification document
- Validation workflow

Testing

- All Python files compile
- All required files present
- vercel.json is valid JSON
- action.yml structure validated

Deployment

Ready to deploy to Vercel. Requires a KV store (Vercel KV or Redis) for score indexing.

Spec

See docs/NTI-CERT-SPEC.md for the full protocol specification.

5. Do NOT auto-merge the PR. Leave it open.

6. Report back with:

- Repo URL
- File count (should be ~27 files)
- Tag confirmation (v1)
- PR URL
- Any errors encountered

7. Do NOT touch jules.yml.